

SHADER DSL · WEBGPU · CROMATINA

Plan de trabajo

Infraestructura GPU y análisis de trayectorias

01 Biblioteca base WebGPU

5-10 días · 9-23 octubre de 2026

Resultado: construir un cálculo GPU y visualizar su salida mediante la nueva biblioteca.

1. Implementar la gestión de adaptador, dispositivo, cola y contexto del canvas.
2. Crear, actualizar y liberar buffers, texturas y samplers.
3. Incorporar pipelines de cálculo y renderizado, bind groups y envío de comandos.
4. Implementar el paso de variables TypeScript a parámetros y recursos GPU, y la lectura asíncrona de resultados.
5. Consultar los límites y capacidades de la GPU e informar de configuraciones incompatibles.

02 Estructuras de datos

3-5 días · 16-30 octubre, en paralelo con la biblioteca

Resultado: utilizar datos de objetos TypeScript en estructuras WGSL desde el DSL.

6. Definir la conversión de objetos y clases TypeScript a estructuras compatibles con WGSL.
7. Declarar tipos, dimensiones y disposición de los campos para el intercambio de datos.
8. Incorporar una sintaxis para declarar estas estructuras sin crear una clase TypeScript.

03 Sintaxis y conexión de programas

4-6 días · 23 octubre-6 noviembre

Resultado: declarar una cadena de cálculo y dibujo que reutilice sus resultados en GPU.

9. Incorporar la selección del backend y una declaración explícita para habilitar las funciones y objetos exclusivos de WebGL2.
10. Declarar programas de cálculo y dibujo con sus entradas, salidas y parámetros.
11. Enlazar la salida de un programa con la entrada del siguiente sin transferencias intermedias a CPU.
12. Verificar la compatibilidad de tipos, dimensiones y acceso entre programas conectados.

SHADER DSL · WEBGPU · CROMATINA

Funciones y ejecución

04 Funciones matemáticas y generación WGSL

5-8 días · 2-13 noviembre

Resultado: definir funciones reutilizables desde archivos, expresiones del DSL y LaTeX.

13. Cargar funciones y programas desde archivos WGSL.
14. Asociar funciones matemáticas con programas y sus entradas.
15. Traducir expresiones y funciones del DSL a WGSL, definiendo las estructuras admitidas.
16. Mostrar errores concretos cuando una expresión utilice operaciones no soportadas.
17. Reutilizar funciones en distintos programas sin duplicar su definición.
18. Incorporar la lectura de expresiones LaTeX desde el DSL y su traducción a WGSL. Estimación específica: 1-2 días.

05 Memoria y ejecución por lotes

5-8 días · 9-20 noviembre

Resultado: ejecutar datos grandes y conectar sus resultados sin reorganizarlos manualmente.

19. Calcular la memoria necesaria y ajustar la ejecución a los límites de la GPU.
20. Dividir las operaciones compatibles en múltiples pasadas y reutilizar los recursos GPU.
21. Mantener los datos en GPU cuando sea posible y gestionar su almacenamiento en RAM cuando sea necesario.
22. Coordinar las dependencias y la sincronización entre pasadas.
23. Resolver bordes y reducciones entre lotes cuando el algoritmo los requiera.
24. Diferenciar las operaciones independientes de las que necesitan datos globales para aplicar una división correcta.
25. Mantener el formato y el orden de las salidas para conectarlas con los siguientes programas.

06 Ejecucion en Node.js

2-4 días · 16-27 noviembre

Resultado: ejecutar la misma cadena de cálculo desde el paquete fuera del navegador.

26. Incorporar y verificar la ejecución de los programas y la exportación de sus resultados en Node.js mediante WebGPU.

Desarrollo matemático

07 Módulos matemáticos

23 noviembre-18 diciembre

Resultado: portar a Shader DSL y WebGPU la cadena matemática del análisis de trayectorias.

27. Generar trayectorias brownianas sintéticas con parámetros y semillas reproducibles. Estimación: 1-2 días.
28. Implementar `KLDivergence` con `D_KL`, tratamiento de densidades nulas y penalización KL del coste. Estimación: 2-3 días.
29. Implementar `MarkovLagScan` con `markov_test_kde` para evaluar la propiedad de Markov y seleccionar el lag temporal por dimensión. Estimación: 3-4 días.
30. Implementar `LeastSquares` para el ajuste inicial y comparar sus coeficientes y residuos con Python. Estimación: 1-2 días.
31. Portar `AdjFP.derivs1d` y construir el operador `diag(f)Dx + diag(a)Dxx` con condiciones de contorno explícitas. Estimación: 2-3 días.
32. Implementar la propagación temporal mediante Krylov/Arnoldi y comparar su acción con `expm(L*tau)` en mallas pequeñas. Estimación: 3-4 días.
33. Implementar `AdjFP.solve` en el módulo `AdjointFP`, integrando operador, propagación y función de coste. Estimación: 2-3 días.
34. Implementar `SteadyFP.solve` en el módulo `StationaryFP`, verificando la distribución estacionaria, su normalización y la convergencia. Estimación: 2-3 días.
35. Implementar `NelderMeadGPU` como módulo de minimización con una función de coste configurable, considerando otras opciones de optimización si son necesarias. Estimación: 5-8 días.
36. Integrar la minimización con el coste del análisis y distribuir su ejecución entre CPU y GPU según corresponda.
37. Verificar las condiciones de contorno, la conservación de probabilidad y la estabilidad numérica de los cálculos Fokker-Planck.

Validación y calendario

08 Validación y aplicación a cromatina

6-9 días de integración · 4-15 enero de 2027

Resultado: obtener resultados reproducibles y contrastados para X e Y por separado.

- 38. Comparar cada función y sus resultados intermedios con Python durante su implementación; utilizar MATLAB cuando exista una referencia concreta.
- 39. Mantener casos conocidos y pruebas de regresión para detectar errores en cada función.
- 40. Validar la cadena completa con trayectorias sintéticas antes de utilizar datos biológicos.
- 41. Registrar entradas, parámetros, semillas y versiones para reproducir los resultados.
- 42. Guardar una ruta de datos de referencia para depuración con salidas de cada etapa.
- 43. Exportar datos y visualizaciones de AFP, KL, costes, distribuciones y selección de lag.
- 44. Medir tiempo y memoria distinguiendo cálculo, transferencias y renderizado.
- 45. Ejecutar el análisis de cromatina en X e Y por separado y comparar las condiciones biológicas disponibles.
- 46. Documentar las discrepancias respecto a Python, las limitaciones y los criterios de validez de los resultados.

09 Calendario de trabajos

Período	Trabajo previsto
9-23 octubre	Biblioteca WebGPU y transferencias de variables
26 octubre-6 noviembre	Estructuras de datos y cadena DSL de cálculo y dibujo
9-20 noviembre	Funciones WGSL, expresiones LaTeX y ejecución por lotes
23-27 noviembre	Node.js, trayectorias sintéticas, KL e inicio del análisis de Markov
30 noviembre-4 diciembre	Markov, mínimos cuadrados y operador adjunto
7-11 diciembre	Propagación Krylov/Arnoldi y soluciones adjunta y estacionaria
14-18 diciembre	Minimización e integración del coste
21 diciembre-3 enero	Carga reducida: correcciones y documentación
4-15 enero	Validación completa, análisis de cromatina y resultados
18-29 enero	Margen para incidencias y cierre documental antes de febrero